

Advanced Algorithms. Assignment 2

Exercise 3.

Note that a network may contain directed edges $e = (v, s)$ that enter the source s , and $f(e) > 0$ is quite possible there.

Now, consider a network with integer edge capacities and a given flow f with integer values $f(e)$ on all edges e . Show that one can construct a flow f' with the following properties: $val(f') \geq val(f)$, and $f'(e) = 0$ on all edges e that enter the source.

Exercise 4.

The following scenario is a bit made up, but it may reflect a real situation in a simplified form. We are given a transportation network, represented by a directed acyclic graph (DAG) $G = (V, E)$ with two distinguished nodes $s, t \in V$ satisfying the following condition:

(R) Every node is reachable from s on some directed path, and t is reachable from every node on some directed path.

Imagine that s is a source of raw materials, the other nodes host several factories, t is some location to store the final products, and the edges are tracks where loaded vehicles can move in only one direction. The network has grown historically. Now, the managers decide to shut down some of the tracks in order to minimize the maintenance costs. But condition (R) must still hold. For simplicity we assume that all tracks have the same costs. Hence, the following problem arises: Delete a maximum number of edges from G while preserving property (R).

The problem seems to be related to flows and cuts, but a successful approach is not completely obvious. We construct a bipartite graph B as follows: Every node $v \neq s, t$ is doubled, that is, we replace v with two copies v_1 and v_0 . But for s and t we only create s_0 and t_1 , respectively. Let V_0 be the set of all nodes v_0 , and let V_1 be the set of all nodes v_1 . For every directed edge (u, v) in G , we create the edge (u_0, v_1) in B .

Now, this bipartite graph is utilized as follows:

- 4.1. Prove that a DAG satisfies (R) if and only if every node $v \neq s$ has at least one incoming edge and every node $v \neq t$ has at least one outgoing edge. (For the proof, remember that every DAG allows a topological ordering, that is, a left-to-right total ordering of the nodes such that every directed edge goes to the right.)
- 4.2. Solve the problem given above by using a maximum matching in B . Most importantly, show the equivalence of our problem to maximum bipartite matching. (You may need the result of 4.1 here.)
- 4.3. Show that the resulting algorithm needs only $O(nm)$ time, when G has n nodes and m edges.